УЧЕБНИК ПО ХОСТИНГУ

Файлы, серверы, Apache, Nginx, PHP

Этот учебник — практическое руководство по работе с файлами и настройке веб-серверов на Ubuntu/Debian. Разбираются: базовые команды Linux, права доступа, архивы, SSH/SCP, конфигурация Apache и Nginx, настройка PHP, кодировки, невидимые dotfiles, SSL/TLS через Let's Encrypt и безопасность сервера.

Все примеры рассчитаны на запуск на VPS или выделенном сервере с Ubuntu 22.04+ и PHP 8.x. На shared-хостинге часть команд (особенно systemctl и установка пакетов) недоступна — там настройка идёт через панель управления.

ОГЛАВЛЕНИЕ

- Глава 1. Введение в хостинг и сервер
- Глава 2. Базовые команды и dotfiles
- Глава 3. Просмотр и редактирование файлов
- Глава 4. Права доступа и ссылки
- Глава 5. Архивы и передача файлов
- Глава 6. Apache
- Глава 7. Nginx
- Глава 8. PHP: конфигурация и интеграция
- Глава 9. Кодировки и dotfiles настроек
- Глава 10. SSH/SCP и безопасность
- Глава 11. SSL/TLS и Let's Encrypt
- Глава 12. Практические задачи и итоги

ГЛАВА 1. ВВЕДЕНИЕ В ХОСТИНГ И СЕРВЕР

1.1. Типы хостинга

Прежде чем углубляться в команды, важно понимать, где именно живёт ваш сайт. Существует три основных типа хостинга, и от этого зависит, какие настройки вам доступны.

- Shared — один сервер на много сайтов
- VPS — виртуальный сервер с root-доступом
- Dedicated — физический сервер полностью ваш

Shared-хостинг — самый дешёвый и простой вариант. Вы арендуете место на сервере вместе с десятками других клиентов. Управление через панель (cPanel, ISPmanager), без доступа к настройкам сервера. Подходит для небольших сайтов и новичков.

VPS (Virtual Private Server) — виртуальный сервер с root-доступом. Вы получаете полную свободу: можно ставить любые пакеты, менять конфиги, перезапускать сервисы. Большая часть этого учебника именно про VPS. Провайдеры: DigitalOcean, Hetzner, Vultr, российские — Timeweb Cloud, Selectel.

Dedicated server — физическая машина целиком ваша. Дорого, для тяжёлых проектов с большими нагрузками. Настройка не отличается от VPS — те же команды, те же конфиги.

1.2. Подключение к серверу

После аренды VPS вы получаете IP-адрес и пароль пользователя root. Подключение через SSH:

```
ssh root@203.0.113.10
# первый вход — спросит отпечаток ключа
# введите yes и пароль
```

В Windows 10+ SSH встроен в командную строку. В старых версиях используют PuTTY. На macOS и Linux ssh доступен из терминала по умолчанию.

Сразу после первого входа рекомендуется создать обычного пользователя и работать от него, а не от root — это безопаснее. Подробнее в главе 10.

1.3. Структура каталогов Linux

Linux имеет древовидную файловую систему с единым корнем /. Нет дисков C: и D: как в Windows. Главные каталоги, которые вам понадобятся:

- /var/www — файлы сайтов
- /etc — конфиги всех сервисов

- /etc/apache2 — конфиги Apache
- /etc/nginx — конфиги Nginx
- /etc/php/8.x — конфиги PHP
- /home/user — домашние папки пользователей
- /root — домашняя папка root
- /tmp — временные файлы
- /var/log — логи
- /usr/share — статические данные пакетов

Ваши сайты живут в /var/www. Обычно каждый сайт — отдельная папка: /var/www/mysite, /var/www/blog и т.д. Конфиги веб-серверов — в /etc. Логи — в /var/log/apache2 или /var/log/nginx.

1.4. Пользователь www-data

Веб-серверы (Apache, Nginx) запускаются от имени специального пользователя www-data (на Ubuntu/Debian). Это сделано для безопасности: если злоумышленник взломает сайт, он получит права www-data, а не root, и не сможет управлять сервером.

Из этого следует важное правило: файлы сайта должны быть читаемы пользователем www-data. Каталоги — читаемы и исполняемы (для входа в них). Папка uploads (куда PHP пишет загруженные файлы) — ещё и доступна на запись. Подробнее о правах в главе 4.

Узнать, от какого пользователя работает веб-сервер:

```
ps aux | grep -E 'apache|nginx'
# www-data  1234 ... apache2
# www-data  1235 ... nginx: worker
```

ГЛАВА 2. БАЗОВЫЕ КОМАНДЫ И DOTFILES

2.1. Навигация по файловой системе

Базовые команды для перемещения:

- pwd — текущий путь (где вы сейчас)
- ls — список файлов в текущей папке

- ls -l — подробный список с правами и размером
- ls -a — вместе со скрытыми (dotfiles)
- ls -lh — размеры в КБ/МБ вместо байтов
- cd /var/www — перейти в каталог сайтов
- cd .. — на уровень выше
- cd ~ — в домашнюю папку
- cd - — вернуться в предыдущую

Чаще всего пользуются комбинацией ls -lah — показать всё подробно, включая скрытые файлы, с человеческими размерами:

```
$ ls -lah /var/www/mysite
total 48K
drwxr-xr-x 4 user www-data 4.0K .
drwxr-xr-x 3 root root     4.0K ..
-rw-r--r-- 1 user www-data 1.2K .env
-rw-r--r-- 1 user www-data  12 .htaccess
drwxr-xr-x 5 user www-data 4.0K assets
-rw-r--r-- 1 user www-data 2.1K index.php
```

2.2. Что такое dotfiles

Dotfiles — это файлы и папки, имя которых начинается с точки. В Linux и macOS они скрыты от обычного ls: чтобы их увидеть, нужно добавить флаг -a. Скрытость не связана с правами доступа — это лишь соглашение оболочки не показывать «технические» файлы в обычном выводе.

Зачем нужны dotfiles в веб-разработке:

- .htaccess — настройки Apache для каталога
- .env — переменные окружения (пароли, ключи)
- .gitignore — список файлов для исключения из Git
- .gitconfig — глобальные настройки Git
- .htpasswd — файл с пользователями и паролями
- .user.ini — настройки PHP для каталога
- .well-known — папка для верификации домена (SSL, домен-валидация)

Это не просто «спрятанные» файлы — это конфигурация вашего проекта и окружения. В отличие от файлов с конфигурацией в /etc, dotfiles лежат рядом с кодом и применяются только к своему каталогу.

2.3. .htaccess — настройка Apache

Файл .htaccess — самый частый dotfile в веб-разработке. Он содержит директивы Apache, применяемые к текущему каталогу и всем подкаталогам. Пример типичного .htaccess в корне сайта:

```
# Запретить вывод листинга каталога
Options -Indexes

# Включить модуль rewrite
RewriteEngine On

# Перенаправление HTTP → HTTPS
RewriteCond %{HTTPS} off
RewriteRule (.*) https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]

# ЧПУ-адреса (красивые URL)
RewriteRule ^article/(\d+)$ article.php?id=$1 [L]

# Запретить доступ к .env и .git
<FilesMatch "^\.">
  Require all denied
</FilesMatch>
```

Важно: .htaccess работает только в Apache, в Nginx его игнорируют. В Nginx все эти правила прописываются в конфиге серверного блока. Кроме того, Apache с AllowOverride None тоже будет игнорировать .htaccess — это иногда делают ради производительности (главный конфиг читается один раз, .htaccess — на каждый запрос).

2.4. Создание и удаление файлов

Базовые команды для манипуляции файлами и папками:

- touch file.txt — создать пустой файл
- mkdir folder — создать каталог
- mkdir -p a/b/c — создать с родителями
- rm file.txt — удалить файл
- rm -r folder — удалить каталог рекурсивно
- rm -rf folder — без подтверждения (ОСТОРОЖНО!)
- cp file.txt copy.txt — копировать
- cp -r folder copy — копировать каталог
- mv old.txt new.txt — переименовать/переместить

Команда rm -rf — самая опасная в Linux. Она удаляет рекурсивно, без подтверждений и не спрашивая. Случайный пробел в пути приведёт к потере данных: rm -rf / var/www удалит корень системы. Перед выполнением проверяйте

путь дважды, особенно с sudo.

Создать сразу несколько dotfiles:

```
cd /var/www/mysite
touch .htaccess .env .gitignore
ls -a  # убедиться, что они созданы
```

ГЛАВА 3. ПРОСМОТР И РЕДАКТИРОВАНИЕ

3.1. Просмотр содержимого файлов

Команды для чтения файлов без редактирования:

- cat file.txt — вывести весь файл
- less file.txt — постранично (q — выход)
- head -n 20 file — первые 20 строк
- tail -n 50 file — последние 50 строк
- tail -f log.txt — следить за концом файла в реальном времени
- wc -l file — сколько строк в файле

tail -f — самая полезная команда для отладки. Пока она запущена, новые строки в логе показываются сразу. Используется для мониторинга ошибок Apache/Nginx/PHP в реальном времени:

```
tail -f /var/log/apache2/error.log
tail -f /var/log/nginx/error.log
tail -f /var/log/php8.3-fpm.log
```

Чтобы остановить tail -f, нажмите Ctrl+C.

3.2. Поиск в файлах — grep

Команда grep ищет текст в файлах. Самые частые флаги:

- grep "ошибка" file.txt — найти строку
- grep -i — без учёта регистра
- grep -r "ошибка" . — рекурсивно в текущей папке
- grep -n — с номерами строк

- grep -v — инвертировать (строки БЕЗ слова)
- grep -E — регулярные выражения

```
# Найти все ошибки в логе
grep -i "error" /var/log/apache2/error.log

# Рекурсивно найти упоминание функции в коде
grep -rn "mysqli_query" /var/www/mysite

# Найти строки БЕЗ комментария (без #)
grep -v "^#" /etc/apache2/apache2.conf
```

3.3. Поиск файлов — find

Команда find ищет файлы по имени, размеру, времени изменения:

- find . -name "*.php" — все PHP-файлы
- find /var/www -name ".env" — найти все .env
- find . -type d — только каталоги
- find . -type f — только файлы
- find . -mtime -7 — изменённые за 7 дней
- find . -size +10M — больше 10 МБ
- find . -name "*.log" -delete — найти и удалить

```
# Найти все .htaccess на сервере
find /var/www -name ".htaccess" 2>/dev/null

# Найти файлы старше 30 дней в бэкапах
find /backup -mtime +30 -name "*.tar.gz"

# 2>/dev/null подавляет ошибки доступа
```

Современная альтернатива find — команда fd (fd-find). Она быстрее, игнорирует .gitignore по умолчанию и имеет человеческий синтаксис. Но find есть везде, а fd — нет.

3.4. Текстовые редакторы vim и nano

На сервере нет графических редакторов, только консольные. Два самых частых — nano (простой) и vim (мощный, но требует обучения).

nano — для новичков. Открывает файл, вы редактируете как в обычном редакторе, сохраняете через Ctrl+O (затем Enter), выходите через Ctrl+X. Подсказки по горячим клавишам видны внизу.

```
nano /etc/apache2/sites-available/mysite.conf
# Ctrl+O → Enter  (сохранить)
# Ctrl+X           (выйти)
# Ctrl+K           (вырезать строку)
# Ctrl+W           (поиск)
```

vim — мощнее, но синтаксис другой. Два режима: командный (по умолчанию) и ввода. Переключение: i (ввод текста), Esc (выход из ввода), :wq (сохранить и выйти), :q! (выйти без сохранения).

```
vim /etc/nginx/sites-available/mysite
# i           — начать ввод текста
# Esc         — выйти из режима ввода
# :wq Enter   — сохранить и выйти
# :q! Enter   — выйти без сохранения
# /слово      — поиск (n — следующее)
# dd          — удалить строку
# yy p        — копировать и вставить строку
```

Если случайно открыли vim и не знаете, как выйти — нажмите Esc, наберите :q! и нажмите Enter. Это спасёт вас от самого частого кошмара новичков.

ГЛАВА 4. ПРАВА ДОСТУПА И ССЫЛКИ

4.1. Что значат права 755 и 644

Linux различает три категории пользователей: владелец (owner), группа (group), остальные (others). Для каждой из них — три права: чтение (r), запись (w), исполнение (x).

В цифровом виде r=4, w=2, x=1. Сумма даёт число от 0 до 7. Запись 755 расшифровывается так: 7=rwx для владельца, 5=r-x для группы, 5=r-x для остальных. То есть владелец может всё, остальные — читать и заходить в каталог, но не писать.

- 7 = rwx — читать, писать, исполнять
- 6 = rw- — читать и писать, не исполнять
- 5 = r-x — читать и исполнять, не писать
- 4 = r-- — только читать
- 0 = --- — ничего нельзя

Стандартные права для сайтов:

- 755 для каталогов (нужно x для входа)

- 644 для обычных файлов (html, php, css, js)
- 600 для файлов с секретами (.env, .htpasswd)
- 644 для конфигов в /etc

4.2. chmod — изменение прав

Команда chmod меняет права. Можно числами или буквами:

```
# Числами
chmod 755 /var/www/mysite
chmod 644 /var/www/mysite/index.php
chmod 600 /var/www/mysite/.env

# Рекурсивно для всех файлов
chmod -R 644 /var/www/mysite

# Рекурсивно, разное для файлов и папок
find /var/www/mysite -type d -exec chmod 755 {} \;
find /var/www/mysite -type f -exec chmod 644 {} \;
```

Последняя конструкция — самая частая в практике. Она выставляет 755 для каталогов и 644 для файлов, что соответствует правильной настройке веб-проекта.

4.3. Почему НЕЛЬЗЯ ставить 777

Права 777 (rwxrwxrwx) — полный доступ для всех. На первый взгляд, простое решение проблемы «файл не записывается». Но это создаёт дыру в безопасности: любой пользователь на сервере может изменить или удалить файл.

Если у вас проблема с записью — её нужно решать через смену владельца (chown) или группы, а не через 777. Типичная ошибка новичков — сделать chmod -R 777 на всю папку сайта. Это открывает возможность подмены файлов PHP и выполнения чужого кода.

4.4. chown — смена владельца

Часто проблема записи в файл — это неправильно назначенный владелец или группа. Команда chown меняет их:

```
# Сделать user владельцем, www-data группой
chown user:www-data /var/www/mysite

# Рекурсивно
chown -R user:www-data /var/www/mysite

# Только для папки uploads (где PHP пишет)
chown -R www-data:www-data /var/www/mysite/uploads
```

Типичная настройка для сайта на Apache/Nginx: владелец файлов — обычный пользователь (вы), группа — www-data. Тогда вы можете редактировать файлы, а веб-сервер — читать их. Папка uploads, куда пишет PHP, принадлежит www-data.

4.5. Символические ссылки — ln -s

Символическая ссылка (symlink) — это «ярлык» на другой файл или каталог. Создаётся командой ln -s:

```
# Синтаксис: ln -s ЦЕЛЬ ИМЯ_ССЫЛКИ
ln -s /var/www/mysite /home/user/site

# Теперь /home/user/site → /var/www/mysite
ls -la /home/user/site
# lrwxrwxrwx user user site -> /var/www/mysite
```

Символические ссылки удобны, когда нужно, чтобы один и тот же файл был доступен из разных мест. Например, общая библиотека для нескольких проектов или конфигурация, переключаемая между версиями:

```
# Несколько версий PHP-конфига
ln -s /etc/php/8.3/apache2/php.ini \
      /etc/php/active.ini

# Чтобы переключиться на 8.2:
rm /etc/php/active.ini
ln -s /etc/php/8.2/apache2/php.ini \
      /etc/php/active.ini
```

В отличие от жёстких ссылок (hard links, ln без -s), символические перескакивают через файловые системы и могут указывать на каталоги. Жёсткие ссылки работают только внутри одного раздела и не могут ссылаться на каталоги. Для большинства задач используют именно символические.

ГЛАВА 5. АРХИВЫ И ПЕРЕДАЧА ФАЙЛОВ

5.1. tar — главный архиватор Linux

tar (tape archive) — стандартный архиватор в Linux. Не сжимает сам, но комбинируется с gzip или bzip2. Основные флаги:

- c — создать архив (create)
- x — извлечь (extract)
- t — посмотреть содержимое (list)
- z — сжать через gzip (.tar.gz)
- j — сжать через bzip2 (.tar.bz2)
- v — подробный вывод (verbose)
- f — имя файла (обязательно последним)

Запомнить мнемонику: c — Create, eXtract, lisT. Флаг f идёт последним и обязательно сопровождается именем файла.

```
# Создать архив сайта
tar -czvf site.tar.gz /var/www/mysite

# Посмотреть, что внутри
tar -tzvf site.tar.gz

# Извлечь в текущую папку
tar -xzvf site.tar.gz

# Извлечь в конкретную папку
tar -xzvf site.tar.gz -C /var/www/
```

Имя архива не обязательно с путём. Часто добавляют дату для бэкапов:

```
tar -czvf site-$(date +%Y-%m-%d).tar.gz \
      /var/www/mysite
# Получится site-2026-10-26.tar.gz
```

5.2. zip и unzip

Формат zip чаще встречается в Windows-мире, но Linux тоже умеет. Удобен, когда нужно передать архив пользователю, который не знаком с tar:

```
# Установить, если нет
apt install zip unzip

# Создать архив
zip -r site.zip /var/www/mysite

# Извлечь
unzip site.zip -d /var/www/
```

5.3. gzip — сжатие одного файла

gzip сжимает один файл, заменяя оригинал на .gz. Часто используется для логов:

```
gzip access.log
# access.log удаляется, появляется access.log.gz

gunzip access.log.gz
# обратная операция
```

5.4. scp — копирование через SSH

Команда scp (secure copy) копирует файлы между компьютерами через SSH. Синтаксис: scp [откуда] [куда]. Удалённые адреса указываются через user@host:путь.

```
# С локального на сервер
scp site.tar.gz root@203.0.113.10:/var/www/

# С сервера на локальный
scp root@203.0.113.10:/var/log/apache2/error.log ./

# Целая папка (-r)
scp -r ./mysite root@203.0.113.10:/var/www/

# С нестандартным портом
scp -P 2222 file.tar.gz root@server:/tmp/
```

scp простой, но не возобновляет разрывы. Для больших файлов и ненадёжных сетей используйте rsync (следующий раздел).

5.5. rsync — синхронизация

rsync — мощный инструмент для синхронизации папок. Переносит только изменившиеся части файлов, поддерживает возобновление и сжатие при передаче. Стандартный набор флагов для бэкапа сайта:

```
rsync -avz --progress \
      /var/www/mysite/ \
      user@backup-server:/backup/mysite/

# -a — архивный режим (сохраняет права, время)
# -v — подробный вывод
# -z — сжатие при передаче
# --progress — показывать прогресс
# --delete — удалять в месте приёма
#             файлы, которых уже нет в источнике
```

Важная тонкость: слэш в конце пути имеет значение. /var/www/mysite/ копирует содержимое папки. /var/www/mysite (без слэша) копирует саму папку. Это источник частых ошибок.

ГЛАВА 6. APACHE

6.1. Установка Apache

На Ubuntu/Debian установка тривиальна через apt:

```
apt update
apt install apache2

# Проверить статус
systemctl status apache2

# Запустить/остановить/перезапустить
systemctl start apache2
systemctl stop apache2
systemctl restart apache2

# Включить автозагрузку
systemctl enable apache2
```

После установки откройте IP сервера в браузере — увидите страницу Apache Ubuntu Default Page. Это значит, сервер работает.

6.2. Структура /etc/apache2

Конфигурация Apache лежит в /etc/apache2. Структура каталога:

- apache2.conf — главный конфиг
- ports.conf — порты (обычно 80 и 443)
- envvars — переменные окружения

- sites-available/ — все доступные сайты
- sites-enabled/ — включенные (символические ссылки)
- mods-available/ — все модули
- mods-enabled/ — включенные модули
- conf-available/ — доп. конфиги
- conf-enabled/ — включенные доп. конфиги

Эта структура available/enabled — особенность Debian-подобных систем. Вместо того чтобы редактировать конфиги вручную, используют вспомогательные команды:

- a2ensite mysite — включить сайт
- a2dissite mysite — выключить сайт
- a2enmod rewrite — включить модуль
- a2dismod rewrite — выключить модуль
- a2enconf charset — включить конфиг
- a2disconf charset — выключить конфиг

Команды создают или удаляют символическую ссылку из available в enabled. После любого изменения нужно перезагрузить Apache:

```
systemctl reload apache2
# reload — мягко, без обрыва соединений
# restart — жёстко, с обрывом
```

6.3. VirtualHost — настройка сайта

VirtualHost — это конфигурация одного сайта. Создаётся файл в sites-available:

```
# /etc/apache2/sites-available/mysite.conf
<VirtualHost *:80>
    ServerName mysite.example.com
    ServerAlias www.mysite.example.com
    DocumentRoot /var/www/mysite

    <Directory /var/www/mysite>
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog  ${APACHE_LOG_DIR}/mysite_error.log
    CustomLog ${APACHE_LOG_DIR}/mysite_access.log combined
</VirtualHost>
```

Здесь:

- ServerName — основной домен

- ServerAlias — дополнительные домены
- DocumentRoot — папка с файлами сайта
- AllowOverride All — разрешить .htaccess
- Require all granted — открыть доступ всем
- ErrorLog/CustomLog — отдельные логи

Включаем сайт и перезагружаем:

```
a2ensite mysite
systemctl reload apache2

# Проверить конфиг на ошибки
apache2ctl configtest
# Syntax OK
```

6.4. Модули Apache

Apache модульная архитектура. Модули добавляют функциональность: rewrite (ЧПУ-адреса), ssl, headers, deflate (сжатие gzip).

```
# Включить mod_rewrite для .htaccess
a2enmod rewrite

# Включить SSL
a2enmod ssl headers

# Перезагрузить
systemctl restart apache2
```

6.5. .htaccess — детали

Файл .htaccess — это точечная настройка Apache для конкретного каталога. Работает только если в главном конфиге разрешено AllowOverride All. Частые примеры:

```
# Кастомная страница 404
ErrorDocument 404 /404.html

# Сжатие gzip
<IfModule mod_deflate.c>
  AddOutputFilterByType DEFLATE text/html
  AddOutputFilterByType DEFLATE text/css
  AddOutputFilterByType DEFLATE application/javascript
</IfModule>

# Кеширование статики
<IfModule mod_expires.c>
  ExpiresActive On
  ExpiresByType image/jpeg "access plus 1 month"
  ExpiresByType text/css "access plus 1 week"
</IfModule>
```

На продакшене .htaccess лучше отключать (AllowOverride None) и переносить все правила в главный конфиг VirtualHost. Это даёт прирост производительности — Apache не ищет .htaccess в каждом каталоге на каждый запрос.

ГЛАВА 7. NGINX

7.1. Установка Nginx

Установка аналогична Apache:

```
apt update
apt install nginx

systemctl status nginx
systemctl enable nginx
systemctl restart nginx
```

Важно: Apache и Nginx по умолчанию слушают порт 80. Нельзя запускать оба одновременно — будет конфликт. Если ставите Nginx поверх Apache, сначала остановите Apache: systemctl stop apache2.

7.2. Структура /etc/nginx

Конфигурация Nginx компактнее Apache:

- nginx.conf — главный конфиг
- sites-available/ — все сайты
- sites-enabled/ — включенные (символические ссылки)

- conf.d/ — доп. конфиги (как conf-enabled у Apache)
- snippets/ — переиспользуемые блоки
- fastcgi_params — параметры FastCGI
- mime.types — соответствие расширений и MIME

7.3. Server block — настройка сайта

Аналог VirtualHost в Nginx называется server block. Создаётся в sites-available:

```
# /etc/nginx/sites-available/mysite
server {
    listen 80;
    server_name mysite.example.com \
                www.mysite.example.com;
    root /var/www/mysite;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    location ~ /\. {
        deny all;
    }

    access_log /var/log/nginx/mysite_access.log;
    error_log  /var/log/nginx/mysite_error.log;
}
```

Здесь:

- listen — порт
- server_name — домены (можно regex)
- root — корневая папка
- index — какие файлы искать по умолчанию
- location / — основной маршрут
- try_files — попытка найти файл, потом каталог, потом index.php
- location ~ \.php$ — обработка PHP через FastCGI
- location ~ /\. — запрет доступа к dotfiles

Включаем сайт:

```
ln -s /etc/nginx/sites-available/mysite \
      /etc/nginx/sites-enabled/

# Проверить конфиг
nginx -t
# nginx: configuration file /etc/nginx/nginx.conf
#        test is successful

# Перезагрузить
systemctl reload nginx
```

Команда nginx -t — обязательная перед reload. Она проверяет синтаксис и спасает от ситуации «перезапустил и сайт упал».

7.4. Отличие Nginx от Apache

Nginx создавался как сервер статики и прокси, Apache — как сервер динамики. Сегодня Nginx быстрее для статических файлов и более эффективен по памяти под высокой нагрузкой. Apache удобнее для разработки (поддержка .htaccess, меньше конфигурации на старте).

Самая частая архитектура: Nginx как фронтенд (раздаёт статику и работает SSL), а PHP-FPM как бэкенд (через fastcgi_pass). Apache в этой схеме не участвует вовсе.

7.5. Запрет доступа к dotfiles в Nginx

В Apache .htaccess-файлы автоматически не отдаются клиентам — Apache скрывает их. В Nginx такой защиты нет по умолчанию, нужно добавить правило вручную:

```
location ~ /\. {
    deny all;
    access_log off;
    log_not_found off;
}
```

Это закрывает доступ ко всем файлам, начинающимся с точки: .env, .git, .htaccess, .htpasswd и т.д. Без этого правила любой может открыть https://example.com/.env и увидеть ваши пароли.

ГЛАВА 8. PHP: КОНФИГУРАЦИЯ

8.1. Установка PHP на Ubuntu

На Ubuntu 22.04+ по умолчанию доступен PHP 8.1. Для более новых версий (8.2, 8.3) добавляют репозиторий PPA:

```
apt update
apt install -y php php-cli php-fpm \
    php-mysql php-mbstring php-xml \
    php-curl php-zip php-gd php-bcmath

# Для новых версий через PPA:
apt install -y software-properties-common
add-apt-repository ppa:ondrej/php
apt update
apt install -y php8.3 php8.3-fpm php8.3-mysql

# Проверить
php -v
php -m | head  # список модулей
```

8.2. PHP как модуль Apache vs PHP-FPM

Есть два способа запускать PHP на сервере:

- mod_php — модуль Apache, встроенный в процесс
- PHP-FPM — отдельный сервис, общается через FastCGI

mod_php проще в настройке (ставится автоматически), но медленнее и ест больше памяти: каждый процесс Apache держит интерпретатор PHP в памяти. Сегодня почти не используется.

PHP-FPM — стандарт для Nginx и рекомендуемый способ для Apache 2.4+. Запускается как отдельный сервис, общается с веб-сервером через FastCGI-протокол. Эффективнее по памяти и быстрее под нагрузкой.

```
# Проверить PHP-FPM
systemctl status php8.3-fpm

# Слушает сокет (обычно):
# /run/php/php8.3-fpm.sock

# Настройка в /etc/php/8.3/fpm/pool.d/www.conf
```

8.3. php.ini — главный конфиг

PHP имеет отдельные php.ini для разных режимов работы:

- /etc/php/8.3/cli/php.ini — для CLI (php script.php)
- /etc/php/8.3/fpm/php.ini — для PHP-FPM (Nginx, Apache+FastCGI)

- /etc/php/8.3/apache2/php.ini — для mod_php

Это разделение часто путает новичков: меняешь в CLI, а в браузере не работает. Важно редактировать именно тот php.ini, который соответствует режиму.

Ключевые настройки для веба:

```
; /etc/php/8.3/fpm/php.ini

; Лимит памяти на один скрипт
memory_limit = 128M

; Максимальный размер загружаемого файла
upload_max_filesize = 64M

; Максимальный размер POST-запроса
post_max_size = 65M

; Лимит времени выполнения (сек)
max_execution_time = 30

; Показывать ошибки (только для разработки!)
display_errors = Off
display_startup_errors = Off

; Логировать ошибки в файл
log_errors = On
error_log = /var/log/php8.3-fpm.log

; Кодировка по умолчанию
default_charset = "UTF-8"
mbstring.internal_encoding = "UTF-8"
```

После изменения php.ini для PHP-FPM нужно перезапустить именно FPM, а не Nginx/Apache:

```
systemctl restart php8.3-fpm
```

8.4. Пользовательский .user.ini

PHP-FPM поддерживает файлы .user.ini — это аналог .htaccess, но для PHP-настроек. Кладётся в каталог сайта и применяется только к нему:

```
# /var/www/mysite/.user.ini
upload_max_filesize = 100M
post_max_size = 105M
display_errors = On
memory_limit = 256M
```

Удобно, когда у вас несколько сайтов с разными требованиями: одному 256 МБ памяти, другому хватит 128. Изменения применяются не сразу, а через user_cache (по умолчанию 300 секунд). Чтобы ускорить — перезапустите PHP-FPM.

8.5. КАК СДЕЛАТЬ .htm файлы PHP-обрабатываемыми

Частая задача: на сайте есть файлы с расширением .htm или .html, и вы хотите, чтобы в них выполнялся PHP-код. По умолчанию Apache и Nginx отдают .htm как статические файлы, без интерпретации.

8.6. .htm как PHP в Apache

В Apache это делается одной директивой в .htaccess или в конфиге VirtualHost:

```
# Вариант 1: AddType (классический)
AddType application/x-httpd-php .htm .html

# Вариант 2: AddHandler (с mod_php)
AddHandler application/x-httpd-php .htm .html

# Вариант 3: для PHP-FPM (через SetHandler)
<FilesMatch "\.(htm|html)$">
    SetHandler "proxy:unix:/run/php/php8.3-fpm.sock|fcgi://localhost"
</FilesMatch>
```

AddType и AddHandler работают с mod_php. Если у вас Apache + PHP-FPM (рекомендуется), нужен SetHandler с proxy — он указывает, что запросы к .htm нужно проксировать в FPM через Unix-сокет.

8.7. .htm как PHP в Nginx

В Nginx конфигурация меняется в server block. Нужно либо расширить существующее location ~ \.php$, либо добавить новое:

```
server {
    # ...

    # Стандартная обработка PHP
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    # Дополнительно: .htm и .html через PHP
    location ~ \.(htm|html)$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
}

# Не забудьте проверить и перезагрузить:
nginx -t &amp;&amp; systemctl reload nginx
```

Проверить, что PHP действительно обрабатывается в .htm-файле, можно создав test.htm:

```
# /var/www/mysite/test.htm
<h1><?php echo 'PHP работает в .htm!'; ?></h1>

# Откройте https://mysite.example.com/test.htm
# Если PHP работает, выведется:
#   PHP работает в .htm!
# Если нет — увидите код PHP как текст
```

ГЛАВА 9. КОДИРОВКИ И DOTFILES НАСТРОЕК

9.1. Что такое кодировка

Кодировка — это способ представить символы текста в виде байтов. На одном и том же байте 0xC3 в Windows-1251 будет буква «Г», в KOI8-R — буква «В», а в UTF-8 — это часть двухбайтовой последовательности другого символа.

Если файл сохранён в одной кодировке, а браузер пытается прочитать его в другой — получаются «кракозябры»: вместо «Привет» видим «Ïðèâåò» или «РµСЂёРІРµС,».

Современный стандарт — UTF-8. Он поддерживает все языки мира, экономно кодирует ASCII (1 байт) и расширяется до 4 байт для редких символов. Используйте его везде и для всего.

9.2. UTF-8 с BOM и без BOM

BOM (Byte Order Mark) — это невидимый символ в начале файла (три байта EF BB BF), который «объявляет» кодировку UTF-8. В Windows-редакторах BOM часто добавляется по умолчанию, в Linux — никогда.

Проблема BOM: PHP интерпретирует его как вывод текста. Если BOM стоит в начале файла, который потом include'ится, этот BOM попадёт в вывод раньше, чем сработает session_start() или header(), и получите ошибку «headers already sent».

Правило: сохраняйте PHP-файлы в UTF-8 без BOM. В большинстве редакторов это настраивается — в VS Code, Sublime, PhpStorm выбирайте «UTF-8», а не «UTF-8 with BOM».

Проверить, есть ли BOM, можно hex-просмотром:

```
head -c 3 file.php | xxd
# 00000000: efbb bf... — есть BOM
# 00000000: 3c3f 70... — нет BOM (это <?p)
```

9.3. Перекодирование файлов — iconv

Если вы получили файлы в Windows-1251, их нужно перекодировать в UTF-8. Команда iconv — стандартный инструмент:

```
# Перекодировать один файл
iconv -f windows-1251 -t utf-8 in.html > out.html

# Пакетно для всех .html в папке
for f in *.html; do
    iconv -f windows-1251 -t utf-8 "$f" > "utf8_$f"
done

# Рекурсивно с find
find . -name "*.html" -exec \
    sh -c 'iconv -f windows-1251 -t utf-8 "$0" > "$0.tmp" &amp;&amp; \
           mv "$0.tmp" "$0"' {} \;
```

iconv не изменяет файл на месте — вывод направляется в новый файл. Чтобы заменить оригинал, используют временный файл и mv. Также доступны флаги -c (игнорировать непереводимые символы) и //IGNORE или //TRANSLIT для целевой кодировки.

9.4. Кодировка в HTML

В HTML кодировка объявляется в <head> через meta-тег:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Заголовок</title>
</head>
```

Современный короткий синтаксис <meta charset="utf-8"> должен идти в первых 1024 байтах документа — браузер сканирует начало файла и перечитывает страницу, если обнаружил другую кодировку.

9.5. AddDefaultCharset в Apache

Apache может сам добавлять HTTP-заголовок Content-Type с указанием кодировки. Директива AddDefaultCharset задаёт её для всех текстовых файлов:

```
# В /etc/apache2/apache2.conf или .htaccess
AddDefaultCharset utf-8

# Или для конкретных типов:
AddCharset utf-8 .html .htm .php .css .js
```

Если ваш сайт в Windows-1251 (исторические проекты), выставляйте windows-1251. Если смешанный — лучше явно указывать charset в каждом .htaccess или в meta-теге.

9.6. Кодировка в Nginx

В Nginx кодировка задаётся директивой charset в server block:

```
server {
    charset utf-8;
    charset_types text/plain text/html \
                  text/css application/javascript;
    # ...
}
```

charset_types задаёт, для каких MIME-типов добавлять ; charset=utf-8 в Content-Type. По умолчанию это только text/html, поэтому для CSS и JS нужно явно расширять список.

9.7. Кодировка в PHP

PHP сам по себе не зависит от кодировки, но работа со строками требует корректных настроек. Расширение mbstring (multi-byte string) обязательно для UTF-8:

```
; /etc/php/8.3/fpm/php.ini

; Кодировка по умолчанию
default_charset = "UTF-8"

; Внутренняя кодировка mbstring
mbstring.language = Russian
mbstring.internal_encoding = "UTF-8"
mbstring.http_input = "UTF-8"
mbstring.http_output = "UTF-8"

; Подмена функций (deprecated в 7.2+!)
; mbstring.func_overload = 7 — НЕ ИСПОЛЬЗОВАТЬ
```

Директива mbstring.func_overload подменяла стандартные строковые функции на их mb_-аналоги. Это создавало проблемы совместимости с библиотеками и объявлена устаревшей. Современный подход — явно вызывать mb_strlen, mb_substr, mb_strtolower и т.д.

9.8. Кодировка в MySQL и PDO

Частый источник кракозябр — разные кодировки у базы и у PHP. Если база в utf8mb4 (правильно), а соединение в latin1 (по умолчанию старого MySQL), строки искажаются при записи и чтении.

В PDO кодировка соединения задаётся через DSN:

```
$dsn = 'mysql:host=localhost;'
       . 'dbname=test;'
       . 'charset=utf8mb4';
$pdo = new PDO($dsn, $user, $pass);

# Альтернатива — через query после подключения:
# $pdo->exec("SET NAMES utf8mb4");
```

Используйте именно utf8mb4, а не utf8 — последний в MySQL обрезает 4-байтные последовательности и не поддерживает эмодзи или редкие китайские иероглифы. utf8mb4 — полноценная UTF-8 в MySQL.

9.9. .env — dotfile с секретами

Файл .env — современный стандарт хранения секретов в проекте. Содержит пароли, ключи API, учётные данные БД — всё, что нельзя коммитить в Git. Структура простая: КЛЮЧ=ЗНАЧЕНИЕ.

```
# /var/www/mysite/.env
DB_HOST=localhost
DB_NAME=mysite
DB_USER=site_user
DB_PASS=S3cretP@ssword
API_KEY=sk_live_abc123
DEBUG=false
```

Чтобы .env был доступен в PHP, его парсят вручную или через библиотеку vlucas/phpdotenv. Простой вариант:

```
$env = parse_ini_file(__DIR__ . '/.env');
define('DB_PASS', $env['DB_PASS']);
```

КРИТИЧЕСКИ ВАЖНО: файл .env должен быть недоступен через веб. В Apache он автоматически скрыт (начинается с точки). В Nginx нужно правило location ~ /\. { deny all; }. Также права на файл — 600, чтобы никто, кроме владельца, не мог читать секреты.

```
# Защита .env
chmod 600 /var/www/mysite/.env
chown user:user /var/www/mysite/.env

# И обязательно в .gitignore:
#.env
```

ГЛАВА 10. SSH/SCP И БЕЗОПАСНОСТЬ

10.1. SSH-ключи вместо пароля

Вход по паролю небезопасен: его можно подобрать брутфорсом, его можно перехватить. SSH-ключи — это пара: закрытый ключ (у вас) и открытый (на сервере). С помощью ключей вы входите без пароля, и подобрать их невозможно.

Создание ключа на локальной машине:

```
ssh-keygen -t ed25519 -C "my@machine"
# Спросит путь (по умолчанию
# ~/.ssh/id_ed25519) и пароль (passphrase)
# на сам ключ — рекомендую задать
```

Современная рекомендация — использовать алгоритм ed25519 вместо старого RSA. Ключ короче, безопаснее, быстрее.

10.2. Копирование ключа на сервер

Чтобы сервер вас узнавал, ваш открытый ключ нужно добавить в файл ~/.ssh/authorized_keys на сервере. Простейший способ — команда ssh-copy-id:

```
ssh-copy-id root@203.0.113.10
# Запросит пароль root один раз
# (последний раз в жизни)
```

Если ssh-copy-id нет (Windows без WSL), можно вручную:

```
# На локальной машине:
cat ~/.ssh/id_ed25519.pub
# Скопировать вывод в буфер

# На сервере:
mkdir -p ~/.ssh
chmod 700 ~/.ssh
nano ~/.ssh/authorized_keys
# Вставить ключ, сохранить
chmod 600 ~/.ssh/authorized_keys
```

10.3. ~/.ssh/config — dotfile для SSH

Файл ~/.ssh/config — удобный dotfile для хранения параметров подключения к разным серверам. Без него приходится помнить IP, порты, имена пользователей. С ним — короткая команда.

```
# ~/.ssh/config (на локальной машине)
Host mysite
    HostName 203.0.113.10
    User deploy
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 60

# Теперь подключение упрощается:
ssh mysite
scp file.tar.gz mysite:/tmp/
rsync -avz ./site/ mysite:/var/www/site/
```

Параметр ServerAliveInterval 60 держит соединение живым, отправляя keepalive-пакет каждую минуту. Полезно, если ваше соединение обрывается при простое (NAT-таймауты, Wi-Fi сети в кафе и т.д.).

10.4. Отключение входа по паролю

После того как SSH-ключ работает, вход по паролю нужно отключить. Это закрывает главный вектор атаки на сервер — подбор пароля ботами (на любом публичном VPS подбор начинается через 5 минут после запуска).

```
# /etc/ssh/sshd_config
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no  # или 'prohibit-password'

# Перезапустить sshd
systemctl restart ssh

# ВАЖНО: не закрывайте текущую сессию,
# пока не проверите в новой вкладке,
# что вход по ключу работает.
# Иначе потеряете доступ к серверу!
```

PermitRootLogin no — запрещает вход пользователем root. Сначала создайте обычного пользователя с sudo-правами, иначе останетесь без админа. Альтернатива — prohibit-password: root может входить только по ключу, но не по паролю.

10.5. Смена порта SSH

Смена стандартного порта 22 на нестандартный (например, 2222) уменьшает шум в логах: 99% ботов проверяют только 22. Это не настоящая безопасность, а гигиена — логи чистые, боты не беспокоят.

```
# /etc/ssh/sshd_config
Port 2222
systemctl restart ssh

# Теперь подключение:
ssh -p 2222 user@server
# Или через ~/.ssh/config:
Host mysite
    Port 2222
```

10.6. fail2ban — защита от перебора

Программа fail2ban анализирует логи и временно банит IP-адреса, с которых идёт слишком много неудачных попыток входа. Стандарт для любого публичного сервера.

```
apt install fail2ban

# Конфиг по умолчанию уже защищает SSH:
# 5 неудачных попыток → бан на 10 минут

# Статус:
systemctl status fail2ban
fail2ban-client status sshd

# Разбанить IP (если ошибочно):
fail2ban-client set sshd unbanip 1.2.3.4
```

10.7. ufw — фаервол

UFW (Uncomplicated Firewall) — упрощённый фронтенд к iptables. Позволяет в две команды закрыть все порты, кроме нужных:

```
# Сброс и базовые правила
ufw default deny incoming
ufw default allow outgoing

# Открыть SSH (или ваш нестандартный порт)
ufw allow 2222/tcp
# Или: ufw allow ssh

# Открыть HTTP и HTTPS
ufw allow 80/tcp
ufw allow 443/tcp

# Включить фаервол
ufw enable
ufw status verbose
```

Правило ufw allow ssh безопасно — оно открывает именно 22 порт. Если вы сменили порт SSH на 2222, ufw allow ssh уже не сработает (ufw не знает про ваш конфиг sshd), нужно явно указывать: ufw allow 2222/tcp.

Важно: не включайте ufw, не открыв порт SSH, иначе потеряете доступ. Всегда проверяйте, что правило для SSH стоит в списке перед ufw enable.

ГЛАВА 11. SSL/TLS И LET'S ENCRYPT

11.1. Зачем HTTPS

HTTPS — это HTTP поверх TLS/SSL. Браузер шифрует трафик между собой и сервером, поэтому злоумышленник на промежуточном узле не может перехватить пароли, куки, содержимое страниц.

Сегодня HTTPS обязателен. Google помечает HTTP-сайты как «незащищённые», понижает их в выдаче. Браузеры ограничивают функционал: без HTTPS не работают service workers, геолокация, куки с SameSite=Strict, HTTP/2 и HTTP/3.

11.2. Let's Encrypt — бесплатные сертификаты

Let's Encrypt — это бесплатный удостоверяющий центр. Выдаёт сертификаты на 90 дней с автоматическим продлением. Никаких платёжек, проверок организации, паспортов — всё работает автоматически через ACME-протокол.

Клиент certbot — стандарт для получения сертификатов. Установка:

```
apt install certbot python3-certbot-nginx
# Для Apache:
apt install certbot python3-certbot-apache
```

11.3. Получение сертификата для Apache

Самый простой способ — вызвать certbot с флагом --apache. Он сам найдёт VirtualHost, запросит сертификат, пропишет пути в конфиг и перезапустит Apache:

```
certbot --apache -d mysite.example.com \
                 -d www.mysite.example.com

# Спросит:
# 1. Email для уведомлений
# 2. Согласие с условиями
# 3. Перенаправлять ли HTTP → HTTPS (yes)
```

После завершения certbot показывает пути к сертификату и ключу. Они обычно лежат в /etc/letsencrypt/live/mysite.example.com/. Apache-конфиг автоматически обновляется: добавляется новый <VirtualHost *:443> с путями к сертификату.

11.4. Получение сертификата для Nginx

Аналогично для Nginx:

```
certbot --nginx -d mysite.example.com \
                -d www.mysite.example.com

# certbot изменит server block:
# - добавит listen 443 ssl;
# - пропишет ssl_certificate и
#   ssl_certificate_key
# - добавит редирект с 80 на 443
```

11.5. Автоматическое продление

Сертификаты Let's Encrypt живут 90 дней. certbot устанавливает systemd-timer, который дважды в день проверяет срок и продлевает сертификаты, которым осталось < 30 дней. Проверить, что таймер работает:

```
systemctl status certbot.timer

# Проверить продление в dry-run режиме:
certbot renew --dry-run
```

При успешном продлении certbot перезагружает Apache или Nginx через deploy-hook (настраивается автоматически). Если вы что-то меняли вручную — проверьте, что в /etc/letsencrypt/renewal-hooks/deploy/ есть скрипт с systemctl reload nginx/apache2.

11.6. Принудительный HTTPS-редирект

Если certbot не добавил редирект автоматически, его можно прописать вручную. В Apache — в .htaccess:

```
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule (.*) https://%{HTTP_HOST}%{REQUEST_URI} \
    [R=301,L]
```

В Nginx — отдельный server block для 80 порта, который только редиректит:

```
server {
    listen 80;
    server_name mysite.example.com \
                www.mysite.example.com;
    return 301 https://$host$request_uri;
}
```

11.7. HSTS — строгая безопасность

Заголовок HSTS (HTTP Strict Transport Security) говорит браузеру: «всегда ходи на этот сайт по HTTPS, даже если пользователь ввёл http://». Защищает от атак понижения версии протокола.

В Apache:

```
Header always set Strict-Transport-Security \
    "max-age=31536000; includeSubDomains"
```

В Nginx:

```
add_header Strict-Transport-Security \
    "max-age=31536000; includeSubDomains" always;
```

Включайте HSTS только после того, как сайт стабильно работает на HTTPS несколько недель. Иначе можно заблокировать себе доступ к сайту, если что-то с HTTPS сломается. max-age начинается с маленького (300), и постепенно увеличивается до года (31536000).

ГЛАВА 12. ПРАКТИЧЕСКИЕ ЗАДАЧИ И ИТОГИ

12.1. Задача 1: бэкап сайта в архив

Сделайте полный бэкап сайта: файлы из /var/www/mysite и дамп базы MySQL.
Решение:

```
# Бэкап файлов с датой
tar -czvf /backup/mysite-$(date +%Y%m%d).tar.gz \
      /var/www/mysite

# Бэкап базы
mysqldump -u root -p mysite_db \
    | gzip > /backup/mysite-db-$(date +%Y%m%d).sql.gz

# Соединить в один архив
tar -czvf /backup/full-$(date +%Y%m%d).tar.gz \
      -C /backup \
      mysite-$(date +%Y%m%d).tar.gz \
      mysite-db-$(date +%Y%m%d).sql.gz

# Удалить бэкапы старше 30 дней
find /backup -name "full-*.tar.gz" \
    -mtime +30 -delete
```

12.2. Задача 2: .htaccess для скрытия dotfiles

Запретите прямой доступ через браузер к файлам .env, .git, .htaccess в Apache. Решение:

```
# /var/www/mysite/.htaccess

# Запретить доступ ко всем dotfiles
<FilesMatch "^\.">
    Require all denied
</FilesMatch>

# Явно запретить .env
<Files ".env">
    Require all denied
</Files>

# Запретить папку .git
<Directory "/var/www/mysite/.git">
    Require all denied
</Directory>
```

12.3. Задача 3: .htm как PHP

Сделайте так, чтобы файлы .htm на вашем сайте обрабатывались как PHP. Сервер — Apache + PHP-FPM. Решение:

```
# /var/www/mysite/.htaccess

<FilesMatch "\.(htm|html)$">
    SetHandler "proxy:unix:/run/php/php8.3-fpm.sock|fcgi://localhost"
</FilesMatch>

# Не забыть:
a2enmod proxy proxy_fcgi
systemctl reload apache2
```

12.4. Задача 4: перекодировка сайта

У вас есть папка со старыми .html файлами в Windows-1251. Перекодируйте всё в UTF-8. Решение:

```
# Проверить текущую кодировку
file -i index.html
# index.html: text/html; charset=iso-8859-1

# Перекодировать все .html рекурсивно
find . -name "*.html" -exec \
    sh -c 'iconv -f windows-1251 -t utf-8 "$0" \
           > "$0.tmp" &amp;&amp; mv "$0.tmp" "$0"' {} \;

# Не забыть заменить meta charset
find . -name "*.html" -exec sed -i \
    's/charset=windows-1251/charset=utf-8/g' {} \;
```

12.5. Задача 5: синхронизация через rsync

Разверните локальную папку ./site на сервер /var/www/mysite. Удалите на сервере файлы, которых уже нет локально. Исключите .git и .env. Решение:

```
rsync -avz --delete \
      --exclude='.git/' \
      --exclude='.env' \
      --exclude='node_modules/' \
      ./site/ \
      deploy@mysite.example.com:/var/www/mysite/

# -a — архивный режим
# -v — подробно
# -z — сжатие при передаче
# --delete — удалять лишнее в приёмнике
# / в конце source — копировать СОДЕРЖИМОЕ
```

Финальный слэш у ./site/ критичен. Без него rsync создаст на сервере /var/www/mysite/site/ — лишний уровень вложенности.

12.6. Задача 6: HTTPS-редирект и HSTS в Nginx

Настройте Nginx: редирект с HTTP на HTTPS + HSTS заголовок. Решение:

```
# /etc/nginx/sites-available/mysite

# HTTP → HTTPS
server {
    listen 80;
    server_name mysite.example.com \
                www.mysite.example.com;
    return 301 https://$host$request_uri;
}

# HTTPS
server {
    listen 443 ssl http2;
    server_name mysite.example.com \
                www.mysite.example.com;

    ssl_certificate     /etc/letsencrypt/live/
                        mysite.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/
                        mysite.example.com/privkey.pem;

    add_header Strict-Transport-Security \
        "max-age=31536000; includeSubDomains" always;

    root /var/www/mysite;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }

    location ~ /\. { deny all; }
}

# Проверка и применение:
nginx -t &amp;&amp; systemctl reload nginx
```

12.7. Задача 7: защита .env и установка прав

Установите правильные права на файлы сайта так, чтобы веб-сервер мог читать всё, но не мог изменять код. Папка uploads доступна на запись. .env — только владелец. Решение:

```
# Владелец — deploy, группа — www-data
chown -R deploy:www-data /var/www/mysite

# Каталоги — 755
find /var/www/mysite -type d -exec chmod 755 {} \;

# Файлы — 644
find /var/www/mysite -type f -exec chmod 644 {} \;

# uploads — запись для www-data
chown -R www-data:www-data /var/www/mysite/uploads
chmod -R 775 /var/www/mysite/uploads

# .env — только владелец (никакого доступа для www-data)
chmod 600 /var/www/mysite/.env
chown deploy:deploy /var/www/mysite/.env

# .htaccess — чтение для всех, изменение только владельцем
chmod 644 /var/www/mysite/.htaccess
```

12.8. Итоги по главам

Глава 1. Хостинг бывает shared, VPS, dedicated. VPS даёт root-доступ и полную свободу. Сайты живут в /var/www, конфиги — в /etc, логи — в /var/log. Веб-сервер работает от пользователя www-data.

Глава 2. Базовые команды: ls -lah, cd, mkdir, rm, cp, mv. Dotfiles — файлы с точкой в начале, скрыты от обычного ls, видны с -a. .htaccess, .env, .gitignore — главные dotfiles веб-разработчика.

Глава 3. cat, less, head, tail -f, grep -r, find. Редакторы: nano (простой), vim (мощный). tail -f — для мониторинга логов в реальном времени.

Глава 4. Права: 755 для каталогов, 644 для файлов, 600 для секретов. Никогда не ставьте 777. chown меняет владельца. Символические ссылки через ln -s — ярлыки на файлы и каталоги.

Глава 5. tar -czvf/-xzvf для архивов, zip/unzip для Windows-формата, gzip/gunzip для одного файла. scp для копирования через SSH, rsync для синхронизации с возобновлением.

Глава 6. Apache: /etc/apache2, sites-available/enabled, a2ensite/a2enmod. VirtualHost описывает сайт. .htaccess — точечная настройка каталога, работает с AllowOverride All.

Глава 7. Nginx: /etc/nginx, sites-available/enabled, server block. location ~ /\. { deny all; } обязательно для скрытия dotfiles. nginx -t перед reload — обязательно.

Глава 8. PHP-FPM — стандарт. php.ini для разных режимов: /cli, /fpm, /apache2. .user.ini — точечная настройка PHP для каталога. .htm как PHP: AddType в Apache, location ~ \.htm$ в Nginx.

Глава 9. UTF-8 — стандарт. Без BOM для PHP. iconv для перекодировки. AddDefaultCharset в Apache, charset utf-8 в Nginx. .env — dotfile с секретами, обязательно chmod 600 и недоступен через веб.

Глава 10. SSH-ключи через ssh-keygen, копирование через ssh-copy-id. ~/.ssh/config — dotfile с параметрами серверов. Отключить PasswordAuthentication. Сменить порт SSH. fail2ban от перебора, ufw — фаервол.

Глава 11. Let's Encrypt — бесплатные сертификаты на 90 дней. certbot --apache/--nginx получает и продлевает автоматически. Редирект 80 → 443, HSTS заголовок для строгой безопасности.

12.9. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, Docker — изолированные окружения, которые работают одинаково на ноутбуке разработчика и на продакшене. Избавляет от «а на моём сервере работает».

Во-вторых, Ansible — автоматизация настройки серверов. Вместо того чтобы вручную ставить пакеты и править конфиги на 10 серверах, вы пишете playbook, который делает это идемпотентно и на всех сразу.

В-третьих, наблюдаемость — логи, метрики, алерты. Loki + Grafana для логов, Prometheus для метрик, Alertmanager для уведомлений. Без этого вы узнаёте о падении сайта от пользователей, а не от системы.

Главный совет: не пытайтесь заучить все команды. Серверное дело — это практика. Арендуйте самый дешёвый VPS за $5 в месяц и разверните на нём свой сайт от начала до конца: установка Apache, PHP, MySQL, получение SSL, настройка бэкапов, мониторинг логов. С каждой новой перенастройкой приходит интуитивное понимание, когда нужен chmod 644, а когда 755, когда tail -f, а когда grep -r. Опыт складывается из десятков настроенных и сломанных серверов, а не из прочитанных справочников.
